Heart Disease Risk Prediction: End-to-End MLOps Pipeline

MLOps Assignment 1 — Final Report

Student Name	KATBA ANKIT CHIMANBHAI
BITS ID	2025AE05229
Course	AIMLCZG523 — Machine Learning Operations (MLOps)
Assignment	Assignment 1
Dataset	UCI Heart Disease — Cleveland subset
Code Repository	https://github.com/2025AE05229/mlops-a1
CI/CD Workflow	https://github.com/2025AE05229/mlops-a1/actions/workflows/ci.yml
Screenshots	https://github.com/2025AE05229/mlops-a1/tree/main/screenshots
Live API (public)	https://heart-disease-api.1s0uu8ax973i.us-south.codeengine.appdomain.cloud Username: admin Password: ce340a68-2574-4e18-928b-f65a4d14a817
Pipeline Video	Walkthrough recording (SharePoint); also in the repository as video-recording.mp4

Table of Contents

1. Setup & Installation Instructions 2
2. Dataset Overview & EDA 2
3. Feature Engineering & Modelling Choices 5
4. Experiment Tracking Summary 7
5. Model Packaging & Reproducibility 8
6. CI/CD Pipeline 8
7. Containerisation 10
8. Architecture Diagram 11
9. Deployment Workflow & Screenshots 12
10. Monitoring & Logging 13

Appendix A. Additional Evidence Screenshots 14

1. Setup & Installation Instructions

All commands are run from the repository root. The project targets **Python 3.11** and was developed on macOS; the same steps work on Linux, and on Windows with the alternative activation command shown.

1.1 Prerequisites

- Python 3.11 or above
- Git
- Docker or Podman (container build and run)
- `kubectl` and `kind` (optional — local Kubernetes deployment)

1.2 Clone & Environment Bootstrap

```
git clone https://github.com/2025AE05229/mlops-a1.git
cd mlops-a1
python3 -m venv .venv
source .venv/bin/activate          # macOS / Linux
# .venv\Scripts\activate           # Windows
pip install -r requirements.txt
```

All 13 packages in `requirements.txt` are pinned to exact versions. A leaner `requirements-serve.txt` (runtime dependencies only) is used inside the Docker image.

1.3 Run the Full ML Pipeline

```
# 1. Download the dataset from UCI (falls back to the bundled copy in data/raw/)
python -m src.ml_pipeline.tasks.download_data
# 2. Clean and preprocess -> data/processed/heart_disease_cleaned.csv
python -m src.ml_pipeline.tasks.preprocess
# 3. Generate EDA figures -> reports/figures/
python -m src.ml_pipeline.tasks.eda
# 4. Tune (GridSearchCV, 5-fold), evaluate and log both models to MLflow
python -m src.ml_pipeline.tasks.evaluate
# 5. Register the champion model in the MLflow Model Registry
python -m src.ml_pipeline.tasks.register_model
# Or the CI-equivalent single entry point (steps 2, 4, 5 + champion.txt)
python src/ml_pipeline/ci_train.py
```

1.4 Run Tests and Linting

```
flake8 src/ tests/
python -m pytest            # 20 tests; JUnit report -> reports/junit.xml
```

1.5 Serve Locally via Docker

```
docker build -t heart-disease-api:local .
docker run --rm -p 8080:8080 \
  -e API_USERNAME=<user> -e API_PASSWORD=<password> heart-disease-api:local
# Smoke test
curl http://localhost:8080/health
```

`podman` can be substituted for `docker` unchanged. `docker compose up --build` is also supported (it requires `API_PASSWORD` to be set in the shell or in a git-ignored `.env` file).

1.6 View the MLflow Experiment Tracking UI

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
# Open http://localhost:5000
```

2. Dataset Overview & EDA

2.1 Dataset

The **UCI Heart Disease — Cleveland subset** is a standard benchmark for heart-disease prediction (source: https://archive.ics.uci.edu/dataset/45/heart+disease). The raw file `processed.cleveland.data` contains

303 patient records described by **13 clinical features** and one multiclass target (0 = no disease, 1–4 = disease grade). It is fetched by `src/ml_pipeline/tasks/download_data.py`, which falls back to the copy stored in `data/raw/` when the download is unavailable. Table 1 lists the features.

Table 1 — Feature descriptions.

Feature	Type	Description
`age`	Numerical	Age in years
`sex`	Categorical	1 = male, 0 = female
`cp`	Categorical	Chest-pain type (1–4)
`trestbps`	Numerical	Resting blood pressure (mm Hg)
`chol`	Numerical	Serum cholesterol (mg/dl)
`fbs`	Categorical	Fasting blood sugar > 120 mg/dl (1 = true)
`restecg`	Categorical	Resting ECG results (0–2)
`thalach`	Numerical	Maximum heart rate achieved
`exang`	Categorical	Exercise-induced angina (1 = yes)
`oldpeak`	Numerical	ST depression induced by exercise
`slope`	Categorical	Slope of the peak exercise ST segment (1–3)
`ca`	Categorical	Number of major vessels coloured by fluoroscopy (0–3)
`thal`	Categorical	Thalassemia (3 = normal, 6 = fixed defect, 7 = reversible defect)
`target`	Binary	0 = no disease, 1 = disease present (after binarisation)

2.2 Data Quality & Preprocessing

- **Missing values:** the raw file encodes them as `?`. Six rows were affected (`ca`: 4 values, `thal`: 2 values). They were imputed with the **median** for numerical columns and the **mode** for categorical columns.
- **Target binarisation:** grade 0 → 0 (no disease), grades 1–4 → 1 (disease present). The raw distribution (0: 164, 1: 55, 2: 36, 3: 35, 4: 13) becomes 164 negative / 139 positive.
- **Types:** all discrete columns were cast to `int`. The cleaned file `data/processed/heart_disease_cleaned.csv` has 303 rows × 14 columns.

Imputation is applied to the full dataset before the train/test split. Because only 6 of 303 records are affected, the effect on the held-out evaluation is negligible.

2.3 Exploratory Data Analysis

All EDA outputs are generated by `src/ml_pipeline/tasks/eda.py` and saved as 300-dpi PNG files in `reports/figures/`: class balance, numerical distributions, categorical relationships and the correlation heatmap. Table 2 summarises the numerical features.

Table 2 — Descriptive statistics for numerical features (N = 303).

Feature	Mean	Std	Min	Median	Max
`age`	54.44	9.04	29	56	77
`trestbps`	131.69	17.60	94	130	200
`chol`	246.69	51.78	126	241	564
`thalach`	149.61	22.88	71	153	202
`oldpeak`	1.04	1.16	0.0	0.8	6.2

Class balance: the dataset is nearly balanced — 164 negative (54.1 %) and 139 positive (45.9 %) records (Figure 1) — so accuracy is a reasonable headline metric alongside ROC-AUC.

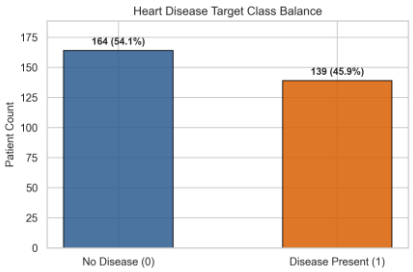


Figure 1 — Target class balance (164 no disease vs 139 disease; N = 303). [View Full Image ↗]

Figure 2 compares the distribution of every numerical feature between the two classes; the categorical features are shown in Figure 21 (Appendix A).

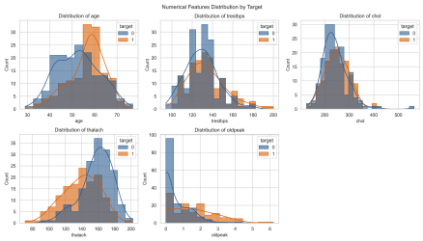


Figure 2 — Distribution of the numerical features by target class. [View Full Image ↗]

2.4 Feature Correlation with Target

Table 3 lists the Pearson correlation of each feature with the binary target (for a binary target this equals the point-biserial correlation), sorted by absolute strength. **thal** (0.522), **ca** (0.460), **exang** (0.432) and **oldpeak** (0.425) are the strongest predictors. **thalach** is inversely correlated (−0.417): a higher maximum heart rate is associated with lower disease risk. Cholesterol (0.085) and fasting blood sugar (0.025) are only weakly related to the target. The full 14 × 14 matrix is shown in Figure 3.

Table 3 — Correlation of each feature with the binary target.

Feature	Correlation (r)	Direction
thal	0.522	Positive
ca	0.460	Positive
exang	0.432	Positive
oldpeak	0.425	Positive
thalach	−0.417	Negative
cp	0.414	Positive
slope	0.339	Positive
sex	0.277	Positive
age	0.223	Positive
restecg	0.169	Weak positive
trestbps	0.151	Weak positive
chol	0.085	Weak positive
fbs	0.025	Negligible

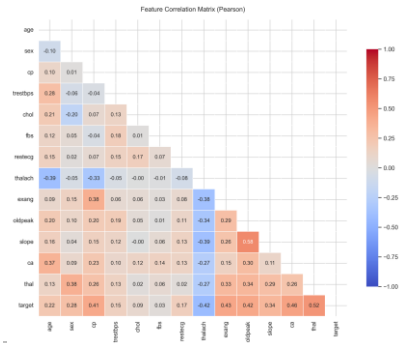


Figure 3 — Pearson correlation matrix of the 13 features and the target. [View Full Image ↗]

3. Feature Engineering & Modelling Choices

3.1 Feature Engineering

All 13 features are retained: with so few variables, each of clear clinical relevance, no feature selection or dimensionality reduction was applied. Preprocessing differs by feature type and is implemented as a Scikit-Learn `ColumnTransformer` (`src/ml_pipeline/tasks/build_features.py`):

- **Numerical pipeline** (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`): `StandardScaler` — zero mean, unit variance. Needed so that Logistic Regression is not dominated by large-scale features such as `chol`.
- **Categorical pipeline** (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`): `OneHotEncoder(handle_unknown="ignore")` — expands each integer-coded category into binary columns so the model does not treat codes as ordered numeric values.

The `ColumnTransformer` and the classifier are combined in a single end-to-end Scikit-Learn `Pipeline`, so the scaler and encoder are fitted only on the training portion of each cross-validation fold, which prevents data leakage. The data is split 80 % / 20 % (stratified, `random_state=42`): N = 242 for training and tuning, N = 61 held out for the final test.

3.2 Candidate Models

Logistic Regression is the primary candidate: it is interpretable, produces well-calibrated probabilities and gives a strong regularised linear baseline once features are scaled.

Random Forest is the non-linear ensemble alternative: it captures feature interactions, is robust to outliers and exposes feature importances, which helps clinical explainability.

3.3 Hyperparameter Tuning

Tuning uses `GridSearchCV` with **Stratified 5-Fold** cross-validation on the training split, optimising `roc_auc` (`n_jobs=-1`, `refit=True`). The Logistic Regression grid has 8 combinations (40 fits) and the Random Forest grid 24 combinations (120 fits). Fixed settings: `random_state=42` for both models and `max_iter=1000` for Logistic Regression.

Table 4 — Hyperparameter search spaces and selected values.

Model	Parameter	Search values	Best value
Logistic Regression	`C`	[0.01, 0.1, 1.0, 10.0]	1.0
Logistic Regression	`solver`	[lbfgs, liblinear]	lbfgs
Random Forest	`n_estimators`	[50, 100, 200]	100
Random Forest	`max_depth`	[3, 5, 8, None]	8
Random Forest	`min_samples_split`	[2, 5]	5

3.4 Evaluation Metrics

Both tuned pipelines were evaluated with 5-fold stratified cross-validation on the training split (Table 5) and once on the 61-patient hold-out test set (Table 6). Values are taken from `reports/model_evaluation.json`.

Table 5 — Cross-validation results (5-fold, mean ± std).

Model	ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Regression ✓	**0.9025 ± 0.0144**	**0.8471 ± 0.0100**	**0.8768 ± 0.0542**	**0.7834 ± 0.0459**	**0.8245 ± 0.0069**
Random Forest	0.9021 ± 0.0217	0.8223 ± 0.0106	0.8282 ± 0.0498	0.7830 ± 0.0741	0.8004 ± 0.0211

Table 6 — Hold-out test-set results (N = 61).

Model	ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Regression ✓	**0.9665**	**0.8852**	**0.8387**	**0.9286**	**0.8814**
Random Forest	0.9502	0.8852	0.8387	0.9286	0.8814

The two models make identical predictions on the test set — both produce the same confusion matrix (TN = 28, FP = 5, FN = 2, TP = 26; see Figure 22 and Figure 23 in Appendix A) — and differ only in their predicted probabilities, hence in ROC-AUC. Figure 4 compares the metrics and Figure 5 and Figure 6 show the ROC curves.

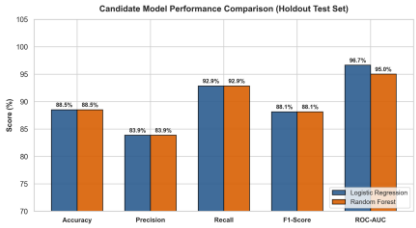


Figure 4 — Hold-out test-set metric comparison of the two candidate models. [View Full Image ↗]

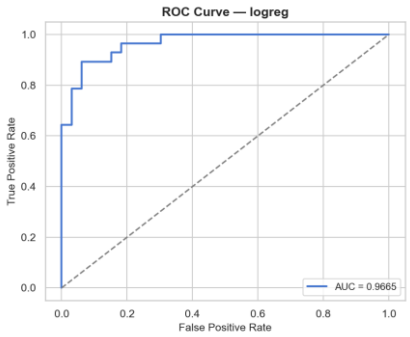


Figure 5 — ROC curve — Logistic Regression (test AUC 0.9665). [View Full Image ↗]

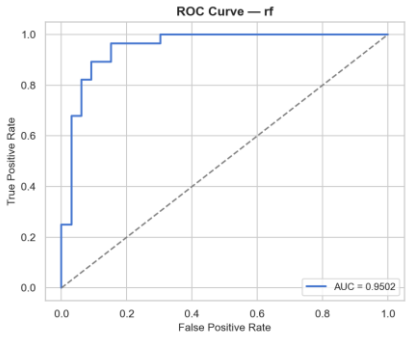


Figure 6 — ROC curve — Random Forest (test AUC 0.9502). [View Full Image ↗]

3.5 Model Selection Decision

Logistic Regression was selected as the champion. Because the two models tie on test accuracy, precision, recall and F1, the decision rests on the remaining evidence:

1. **Higher ROC-AUC** — 0.9665 vs 0.9502 on the test set, and marginally higher cross-validated ROC-AUC (0.9025 vs 0.9021) with lower fold-to-fold variance (± 0.0144 vs ± 0.0217).
2. **Higher cross-validated accuracy** — 84.71 % vs 82.23 %, again with similar variance.
3. **Simplicity and interpretability** — coefficients are directly interpretable as log-odds, the serialised pipeline is 4 KB (vs 553 KB for the forest) and inference cost is negligible, which suits a small containerised microservice.

Recall is the clinically important metric (a missed disease case is the costly error); it is equal for both models at 92.86 % (26 of 28 diseased patients detected). With only 61 test patients, one prediction changes accuracy by about 1.6 percentage points, so test-set differences should be read as indicative; the cross-validation figures are the more reliable guide.

4. Experiment Tracking Summary

4.1 Tracking Backend

MLflow 3.16.1 is used for all experiment tracking, backed by a local SQLite store (`mlflow.db`). Runs are logged to the experiment **`heart-disease-classification`**; the tracking URI is set to `sqlite:///mlflow.db` at the start of every training run. Figure 24 shows the experiments in the UI.

4.2 What is Logged

`mlflow.sklearn.autolog` is enabled with `max_tuning_runs=0`, which keeps all GridSearch artefacts on the parent run instead of spawning one child run per candidate. For every model run it captures:

- **Parameters:** the best hyperparameter combination found by `GridSearchCV`.
- **Metrics:** the best cross-validated ROC-AUC.
- **Model artefact:** the fitted Scikit-Learn `Pipeline` (preprocessing + classifier) with a model signature.
- **CV results table:** full cross-validation scores for every candidate.
- **Dataset inputs:** the train and test DataFrames, logged with `mlflow.log_input`.

After fitting, the evaluation module resumes the same run and additionally logs:

- **Metrics:** test-set accuracy, precision, recall, F1 and ROC-AUC, plus `cv_<metric>_mean/std`.
- **Plots:** a ROC-curve PNG and a confusion-matrix PNG for each model.
- **Summary artefacts:** the model-comparison chart and `model_evaluation.json`, attached to the last (`rf`) run.

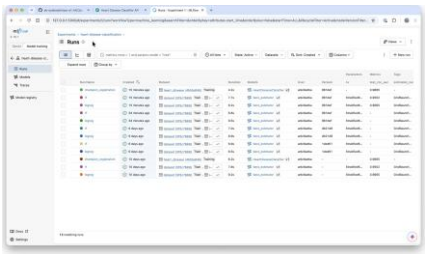


Figure 7 — MLflow Runs view: `logreg`, `rf` and `champion_registration` runs with test ROC-AUC, dataset inputs and registered-model links. [View Full Image ↗]

4.3 MLflow Model Registry

After evaluation, `register_model.py` reads `reports/model_evaluation.json`, selects the model with the highest cross-validated ROC-AUC, and logs that pipeline (with an inferred input/output signature) in a `champion_registration` run. It is registered as a new version of the **`HeartDiseaseClassifier`** model; Figure 8 shows Version 1 and Version 2, the latter created by a later re-run, and Figure 26 shows the 13-column input signature of Version 2. The winning model type is also written to `models/champion.txt` so the serving API can resolve it at start-up without querying MLflow.

Table 7 — MLflow runs in the `heart-disease-classification` experiment.

Run name	Best CV ROC-AUC	Test ROC-AUC	Key artefacts / content
`logreg`	0.9025	0.9665	Pipeline model, CV results, ROC PNG, confusion-matrix PNG, datasets
`rf`	0.9021	0.9502	Pipeline model, CV results, ROC PNG, confusion-matrix PNG, comparison chart, JSON report
`champion_registration`	0.9025	0.9665	Registered `HeartDiseaseClassifier` pipeline with signature; tag `champion_model_type`

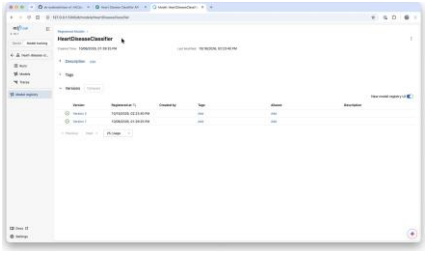


Figure 8 — MLflow Model Registry: `HeartDiseaseClassifier`, Version 1 and Version 2. [View Full Image ↗]

Note: The CI pipeline uploads the `mlruns/` artifact store as the workflow artefact `mlflow-run-<sha>` (30-day retention); run metadata lives in `mlflow.db`.

5. Model Packaging & Reproducibility

5.1 Serialisation Format

Both trained models are saved as Joblib files (`models/logreg_model.joblib`, `models/rf_model.joblib`). Each file contains a full end-to-end Scikit-Learn `Pipeline`:

```
Pipeline
├── preprocessor (ColumnTransformer)
│   ├── num → StandardScaler   (age, trestbps, chol, thalach, oldpeak)
│   └── cat → OneHotEncoder    (sex, cp, fbs, restecg, exang, slope, ca, thal)
└── classifier (LogisticRegression / RandomForestClassifier)
```

Calling `pipeline.predict(raw_df)` on a new raw DataFrame applies exactly the scaling and encoding fitted during training — no separate scaler file and no manual feature transformation are needed at inference time. The champion is additionally registered in the MLflow Model Registry (Section 4.3).

5.2 Active Model Resolution

The serving API resolves the active model at start-up using the following priority chain:

1. `MODEL_PATH` environment variable — explicit override.
2. `models/champion.txt` — written by `ci_train.py` after every training run; contains `logreg` or `rf`.
3. Hard-coded fallback: `logreg`.

`ci_train.py` chooses the champion by highest cross-validated ROC-AUC; this criterion selects Logistic Regression for the current data.

5.3 Dependency Pinning

`requirements.txt` pins its 13 direct dependencies to exact versions (==), so the environment can be recreated from a clean Python 3.12 installation. A separate `requirements-serve.txt` is used in the Docker image; it contains only the runtime packages (NumPy, pandas, scikit-learn, FastAPI, uvicorn, Pydantic and `prometheus-fastapi-instrumentator`) and excludes training, testing and plotting libraries to keep the image small.

5.4 Reproducibility Guide

`REPRODUCIBILITY.md` in the repository root documents:

- Step-by-step environment setup and pipeline re-run commands.
- How to recreate the MLflow experiment from scratch.
- How to load a registered model with `mlflow.sklearn.load_model`.
- How to re-run the test suite.

Randomness is fixed throughout (`random_state=42` for the train/test split, cross-validation folds and both models), so retraining on the same data reproduces the reported metrics.

6. CI/CD Pipeline

6.1 Technology

GitHub Actions (`.github/workflows/ci.yml`, workflow name *MLOps CI Pipeline*) is the CI/CD engine. It triggers on every push and pull request to `main` and runs on `ubuntu-latest` runners with Python 3.11. Each job caches pip downloads keyed on the hash of `requirements.txt`.

6.2 Pipeline Jobs

The workflow has three jobs. `lint` and `test` run in parallel; `train` declares `needs: [lint, test]`, so it starts only when both succeed and a code or test error stops the pipeline before training (Figure 10, Figure 11).

Table 8 — CI/CD pipeline jobs.

Job	Tool	Scope	Gate
`lint`	flake8	`src/` and `tests/` (`setup.cfg`: max line length 120)	`train` cannot run if lint fails
`test`	pytest	20 unit tests across data, model and API layers; uploads `junit.xml`	`train` cannot run if tests fail
`train`	`ci_train.py`	Full pipeline: preprocess → tune → evaluate → register; uploads models and MLflow data	—

6.3 Uploaded Artifacts

Three GitHub Actions artifacts are uploaded with 30-day retention:

- `test-results-<sha>` — JUnit XML (`reports/junit.xml`); uploaded even when tests fail (`if: always()`), so failing runs still carry logs.
- `trained-models-<sha>` — Joblib model files (`models/`).
- `mlflow-run-<sha>` — MLflow artifact store (`mlruns/`).

6.4 Unit Tests

The test suite (`tests/`, configured in `setup.cfg`) contains **20 tests** in three modules. `pytest` writes a JUnit report to `reports/junit.xml` on every run.

Table 9 — Unit-test modules.

Module	Tests	What is verified
`test_data_pipeline.py`	4	Data acquisition, raw-data loading, cleaning (imputation, binary target, integer types), EDA figure generation
`test_model_pipeline.py`	4	Preprocessor, pipeline build and training, prediction shape and probability range, metric evaluation, MLflow run logging during tuning
`test_api.py`	12	`/health`; valid `/predict` payload; 422 for a missing field; 401 (with `WWW-Authenticate`) for missing or wrong credentials on `/predict` and `/metrics`; authenticated `/metrics`

6.5 Pipeline Run History & Failure Evidence

Figure 9 shows the run history of the workflow. Run **#8** ("added authentication on api endpoints") demonstrates the fail-fast behaviour: both the lint job and the unit-test job failed, so the `Model Training` job was skipped and only the `test-results` artefact was uploaded (Figure 11). The pytest log (Figure 12) shows `2 failed, 9 passed`: after authentication was added, two API tests still expected the unauthenticated behaviour (`401 != 422` and `401 != 200`) and the step exited with code 1. The follow-up commit "fix lint and unit-tests on auth code" produced the green run **#10** (Figure 10) with all three artefacts.

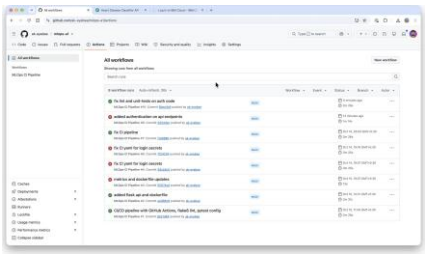


Figure 9 — GitHub Actions run history of *MLOps CI Pipeline* (green = passed, red = failed). [View Full Image ↗]

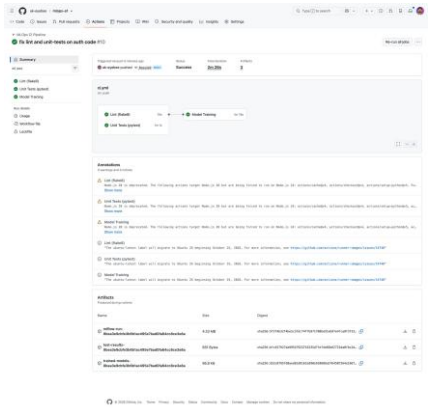


Figure 10 — Run #10 (success): lint and unit tests in parallel, then Model Training; three artefacts. [View Full Image ↗]

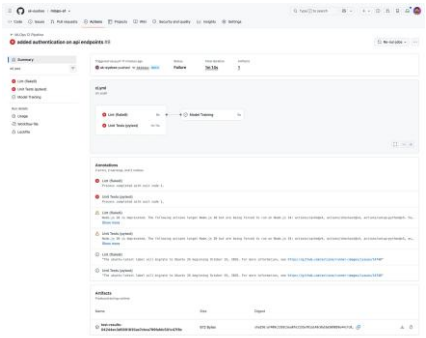


Figure 11 — Run #8 (failure): lint and unit-test jobs failed; Model Training skipped. [View Full Image ↗]

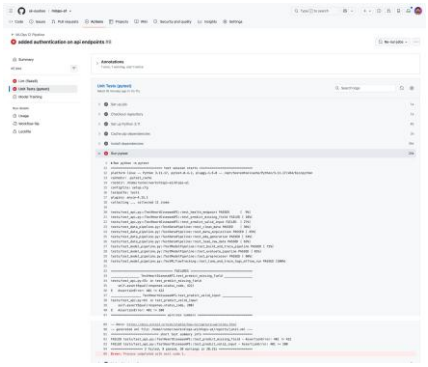


Figure 12 — Run #8 pytest log: two failed API tests (`2 failed, 9 passed`), step exits with code 1. [View Full Image ↗]

7. Containerisation

7.1 Docker Image

The `Dockerfile` builds a lean production image from `python:3.12-slim` (`linux/amd64` platform target). The build:

1. Copies and installs `requirements-serve.txt` (no MLflow, plotting or test packages).
2. Copies the application source (`src/`) and the pre-trained model artefacts (`models/`).
3. Sets `PORT=8080`, exposes port 8080 and starts `uvicorn` through `sh -c` so that `${PORT}` is expanded at run time, allowing IBM Code Engine to inject its own port.

7.2 API Design

The serving application is a FastAPI app in `src/api/app.py`. It exposes the endpoints in Table 10; request and response bodies are validated by Pydantic models.

Table 10 — API endpoints.

Endpoint	Method	Auth	Description
`/health`	GET	None	Liveness probe; returns `{"status":"ok","model":"logreg"}`
`/predict`	POST	HTTP Basic	Accepts a 13-feature JSON body; returns `{"prediction": 0|1, "confidence": <float>}` (probability of disease)
`/metrics`	GET	HTTP Basic	Prometheus-format metrics (request counts, latency histograms)
`/docs`	GET	None	Auto-generated Swagger UI (OpenAPI schema at `/openapi.json`)

7.3 Authentication

HTTP Basic authentication is enforced on `/predict` and `/metrics` and compares credentials with `secrets.compare_digest` to prevent timing attacks. Credentials come from the `API_USERNAME` and `API_PASSWORD` environment variables: in Kubernetes they are injected from a Secret, in Code Engine from the application's environment variables.

7.4 Sample Request

```
curl -u <user>:<password> -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
       "thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
# Response: {"prediction": 0, "confidence": 0.1655}
```

`confidence` is the model's probability for class 1 (disease present). A higher-risk patient (57 years, oldpeak 0.6, exang 1, `thalach` 112, …) returns `{"prediction": 1, "confidence": 0.888}` on the live deployment (Figure 17).

7.5 Container Build & Run Evidence

The image builds from a clean checkout and runs locally. Figure 13 shows the container serving requests, with one structured JSON log line per request, and Figure 14 shows a successful `POST /predict` through the Swagger UI (HTTP 200 with prediction and confidence). The build log, image list, `/health` and `/metrics` calls are in Appendix A (Figure 27 – Figure 30). The image targets `linux/amd64`, so on an Apple-Silicon host Docker prints a platform warning and runs it under emulation.



Figure 13 — Local container started with `docker run`; JSON request logs for `/health`, `/docs`, `/metrics` and `/predict`. [View Full Image ↗]

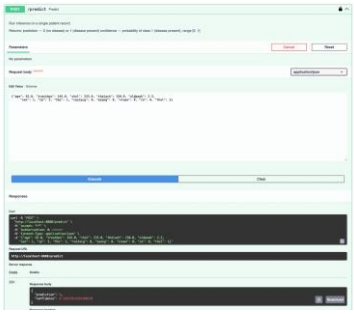


Figure 14 — Swagger UI `POST /predict` on the local container: HTTP 200 with `prediction` and `confidence` (Authorization header redacted). [View Full Image ↗]

8. Architecture Diagram

Figure 15 shows the end-to-end system, from developer commit to production monitoring. A push to `main` triggers GitHub Actions (lint → test → train). The ML pipeline (data → features → tuning → MLflow tracking → joblib pipeline + `champion.txt`) is packaged into a FastAPI Docker image, which is deployed to IBM Code Engine (public HTTPS) and to a local kind cluster behind Contour. Clients call `/health`, `/predict` and `/metrics`; request logs go to stdout and Prometheus metrics feed a Grafana Cloud dashboard.

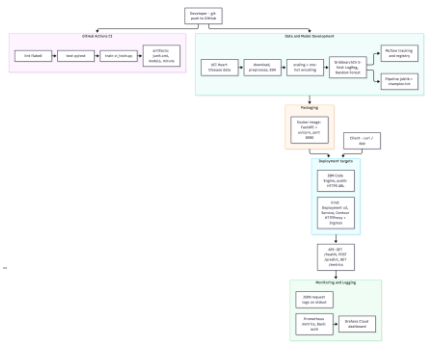


Figure 15 — End-to-end MLOps system architecture. [View Full Image ↗]

9. Deployment Workflow & Screenshots

9.1 Overview

The trained model is served through two deployment targets: a fully public cloud deployment on **IBM Code Engine**, and a local **Kubernetes** cluster created with **kind** and exposed through the **Contour** ingress controller.

9.2 IBM Code Engine (Public Cloud)

1. The Docker image is built locally and pushed to IBM Container Registry (`icr.io`).
2. A Code Engine application `heart-disease-api` is created from the image with port 8080 and the `API_USERNAME` / `API_PASSWORD` environment variables.
3. Code Engine provides a public HTTPS URL automatically: `heart-disease-api.1s0uu8ax973i.us-south.codeengine.appdomain.cloud`.
4. `/health` is open; `/predict` and `/metrics` require HTTP Basic authentication.

The application runs with 1 vCPU / 4 GB per instance, autoscaling between 0 and 1 instance, a target concurrency of 100 and a 300-second request timeout; revision `heart-disease-api-00004` receives 100 % of the traffic (Figure 16, Figure 31). The live endpoint was verified with the Swagger UI (Figure 32) and a direct prediction request (Figure 17).

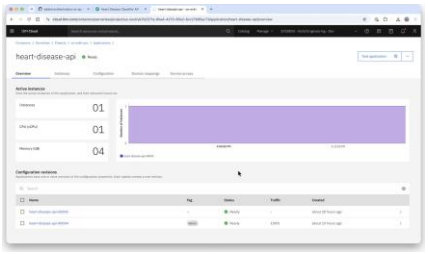


Figure 16 — IBM Code Engine application `heart-disease-api` — Ready; revision 00004 serves 100 % of traffic. [View Full Image ↗]

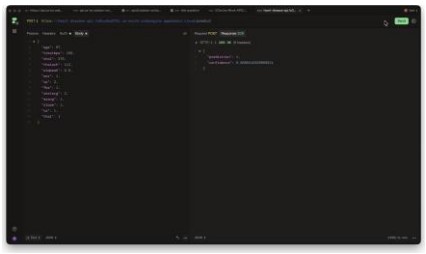


Figure 17 — Live `POST /predict` on the public Code Engine URL: HTTP 200, prediction 1, confidence 0.888. [View Full Image ↗]

9.3 Kubernetes — kind + Contour (Local)

The `deployment/` directory contains a Kustomize-managed set of manifests (Table 11). They create the `heart-disease` namespace, a 2-replica Deployment with probes and resource limits, a ClusterIP Service, a Secret for the credentials and two ways of exposing the service externally: a Contour `HTTPProxy` and a standard `Ingress`.

Table 11 — Kubernetes manifests (`deployment/`).

File	Resource	Key settings
`namespace.yaml`	Namespace	`heart-disease`
`deployment.yaml`	Deployment	2 replicas; RollingUpdate (maxSurge 1, maxUnavailable 0); startup, readiness and liveness probes on `/health`; CPU 100m–500m; memory 256Mi–512Mi
`service.yaml`	ClusterIP Service	Port 80 → container port 8080
`secret.yaml`	Secret	`API_USERNAME`, `API_PASSWORD` injected as environment variables
`httpproxy.yaml`	Contour HTTPProxy	Routes `heart-disease.localtest.me` to the Service
`ingress.yaml`	Ingress	Standard Kubernetes Ingress for `heart-disease-ingress.localtest.me`
`kustomization.yaml`	Kustomize root	Single-command apply: `kubectl apply -k deployment/`

9.4 Deployment Commands

```
# Build the image
podman build -t heart-disease-api:local .
# Load it into the kind cluster
podman save -o /tmp/heart-disease-api.tar localhost/heart-disease-api:local
kind load image-archive /tmp/heart-disease-api.tar --name heart-disease-mlops
# Apply the manifests
kubectl apply -k deployment/
kubectl -n heart-disease rollout status deployment/heart-disease-api
# Access via port-forward
kubectl -n heart-disease port-forward svc/heart-disease-api 8080:80
# Or via the Contour route (kind maps node port 80 -> host port 9090)
curl http://heart-disease.localtest.me:9090/health
```

Figure 18 and Figure 19 show the Deployment with 2/2 pods available and the two running pods. The Service, Ingress, Secret and cluster overview are in Appendix A (Figure 33 – Figure 36).

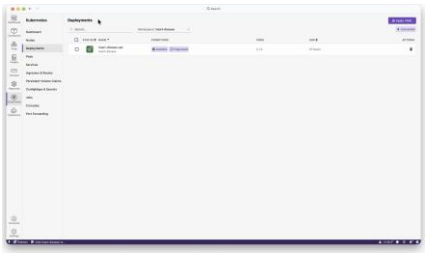


Figure 18 — Deployment `heart-disease-api`: 2/2 pods available. [View Full Image ↗]

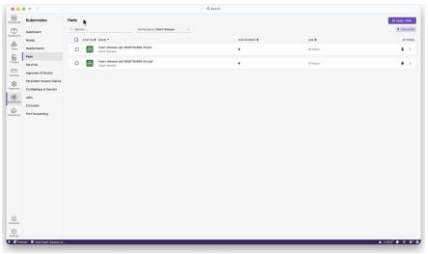


Figure 19 — Two running pods in namespace `heart-disease`. [View Full Image ↗]

10. Monitoring & Logging

10.1 Structured Request Logging

An ASGI middleware in `src/api/app.py` intercepts every HTTP request and emits one JSON line to stdout. Prediction events are additionally logged with the returned class and confidence (sample taken from Figure 13):

```
{"timestamp": "2026-10-15T10:07:30.211918+00:00", "method": "GET", "path": "/docs",
 "status_code": 200, "latency_ms": 11.08}
{"event": "prediction", "prediction": 0, "confidence": 0.1854}
```

In Code Engine, stdout is collected by **IBM Cloud Logs** (Figure 37); in Kubernetes it is captured by the container runtime and can be read with `kubectl logs`.

10.2 Prometheus Metrics

`prometheus-fastapi-instrumentator` instruments all FastAPI routes. The `/metrics` endpoint (protected by HTTP Basic authentication so traffic data is not publicly exposed) returns the standard Prometheus text format (Figure 38). Key metrics:

- `http_requests_total` — counter by method, handler and status code.
- `http_request_duration_seconds` — latency histogram per handler and method.
- Default Python process and garbage-collector metrics.

10.3 Grafana Cloud Dashboard

A Grafana Cloud **Metrics Endpoint** integration scrapes the live API's `/metrics` endpoint every minute with Basic authentication (Figure 39). The *Heart Disease API Dashboard* (Figure 20) shows the success rate, requests per second, total requests, traffic by HTTP status and method, 95th-percentile latency, a latency heatmap, p50/p90/p99 latency and the 5xx error rate (no 5xx responses were recorded).

`scripts/populate_grafana.py` generates synthetic `/predict` traffic — a mix of high- and low-risk patient payloads with jittered timing — so the dashboard shows a meaningful time series.



Figure 20 — Grafana Cloud dashboard "Heart Disease API Dashboard" (20,604 total requests; 100 % success rate). [View Full Image ↗]

Appendix A. Additional Evidence Screenshots

Supporting figures referenced from the sections above.

A.1 EDA and model evaluation

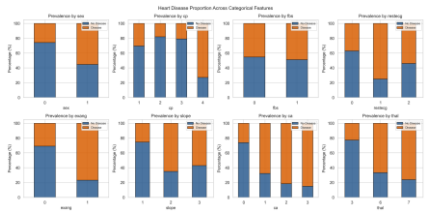


Figure 21 — Disease prevalence across the categorical features. [View Full Image ↗]

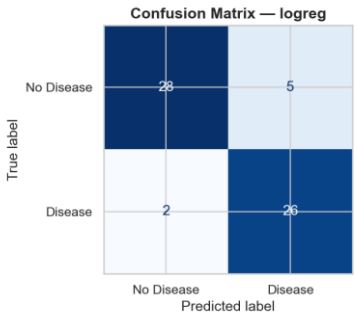


Figure 22 — Confusion matrix — Logistic Regression (test set). [View Full Image ↗]

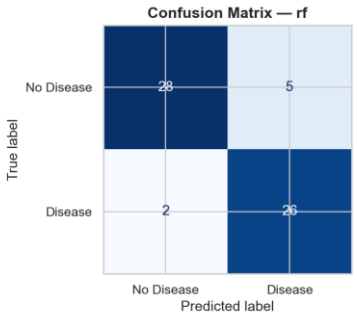


Figure 23 — Confusion matrix — Random Forest (test set). [View Full Image ↗]

A.2 Experiment tracking

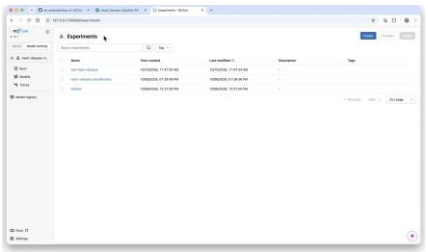


Figure 24 — MLflow Experiments list: `heart-disease-classification` and `test-heart-disease` (created by the unit tests). [View Full Image ↗]

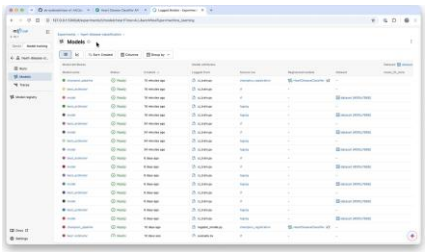


Figure 25 — Logged models of the experiment: per-run `model` and `best_estimator`, and the registered `champion_pipeline`. [View Full Image ↗]

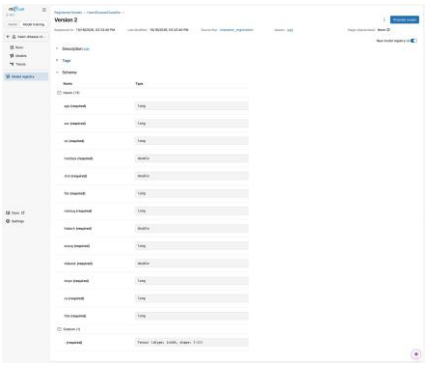


Figure 26 — Registered model Version 2: input/output signature (13 inputs, 1 output). [View Full Image ↗]

A.3 Container and API



Figure 27 — `docker build -t heart-disease-api:local .` completing successfully. [View Full Image ↗]



Figure 28 — `docker images` and `docker ps`: the 514 MB image and the running container on port 8080. [View Full Image ↗]

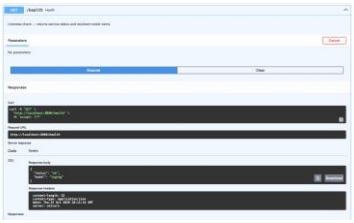


Figure 29 — Swagger UI `GET /health` on the local container (HTTP 200). [View Full Image ↗]

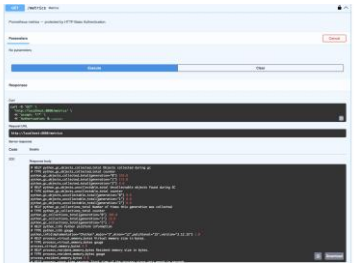


Figure 30 — Swagger UI `GET /metrics` (Basic auth, header redacted): Prometheus text output. [View Full Image ↗]

A.4 Cloud deployment

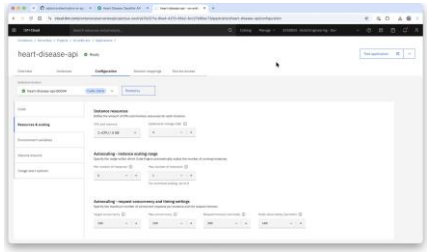


Figure 31 — Code Engine configuration: 1 vCPU / 4 GB, 0–1 instances, concurrency 100, 300 s timeout. [View Full Image ↗]

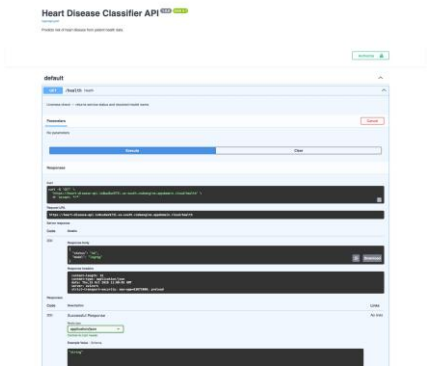


Figure 32 — Swagger UI on the public URL: `GET /health` returns `{"status":"ok","model":"logreg"}`. [View Full Image ↗]

A.5 Local Kubernetes

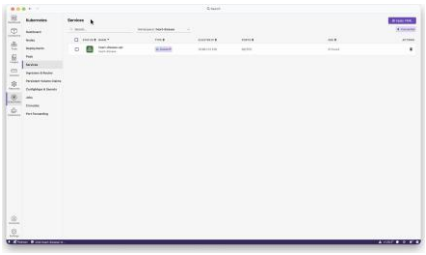


Figure 33 — ClusterIP Service `heart-disease-api` (80/TCP). [View Full Image ↗]

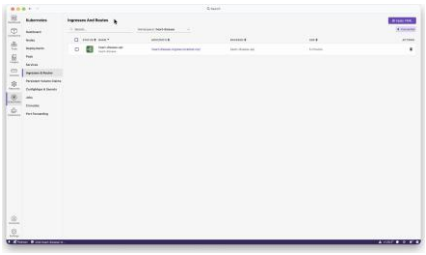


Figure 34 — Ingress for `heart-disease-ingress.localtest.me`. [View Full Image ↗]

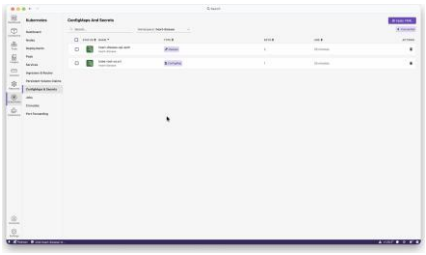


Figure 35 — Secret `heart-disease-api-auth` (Opaque, 2 keys). [View Full Image ↗]

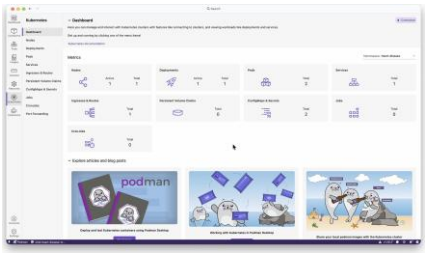


Figure 36 — Cluster overview: 1 node, 1 deployment, 2 pods, 1 service, 1 ingress. [View Full Image ↗]

A.6 Monitoring and logging

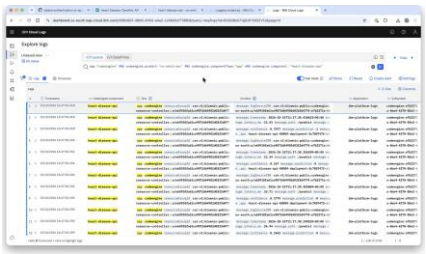


Figure 37 — IBM Cloud Logs: JSON request and prediction log lines from the Code Engine application. [View Full Image ↗]

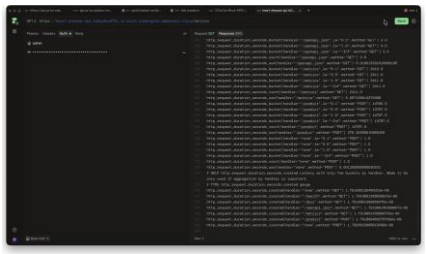


Figure 38 — Live `GET /metrics` response (HTTP 200, Basic auth) with per-handler latency histograms. [View Full Image ↗]

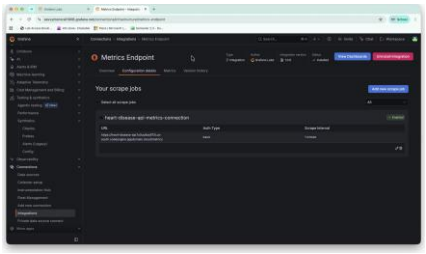


Figure 39 — Grafana Cloud Metrics Endpoint integration: scrape job for the live `/metrics` URL (Basic auth, 1-minute interval). [View Full Image ↗]

— End of Report —